

人工神經網路入門

第 11 週 | 從 Perceptron 到 MLP，走完一次前向與反向傳播

學習重點與成果

- 人工神經元、Perceptron、XOR 與 MLP。
- 非線性、鏈式法則與反向傳播。
- 迴歸／分類架構、訓練與調參。

成果：能列出網路形狀、手算梯度，完成小型 MLP 比較。

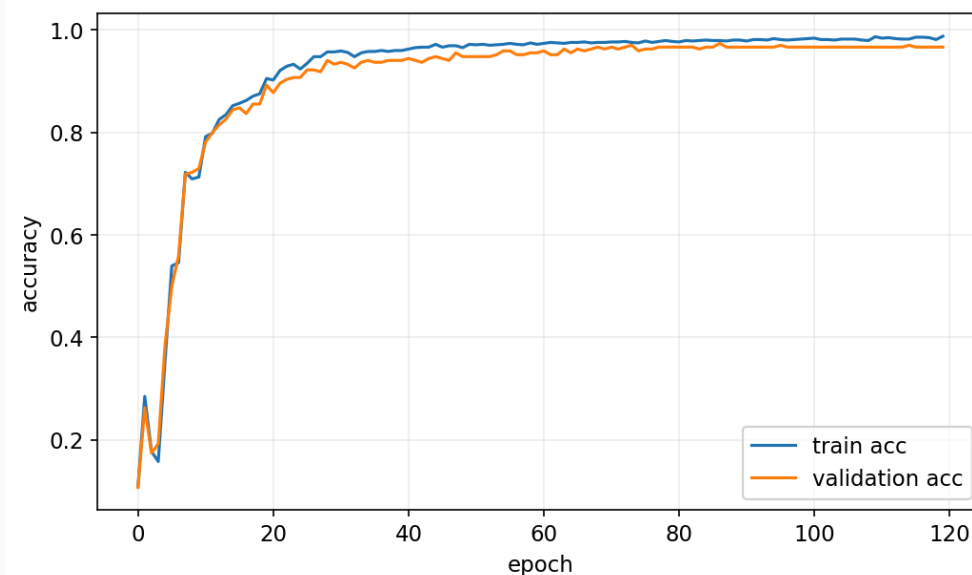
 **配套 Notebook**： `programs/notebooks/11_人工神經網路入門.ipynb`。程式片段、實際圖表與表格可由此檔重現。

程式實驗與實際輸出

```
net.step(X_batch, y_batch, lr=0.5)
validation_acc = mean(net.predict(X_val) == y_val)
```

觀察：同時畫訓練與驗證曲線，才能區分尚未學會與已開始過度擬合。

參考程式：`programs/notebooks/11_人工神經網路入門.ipynb`

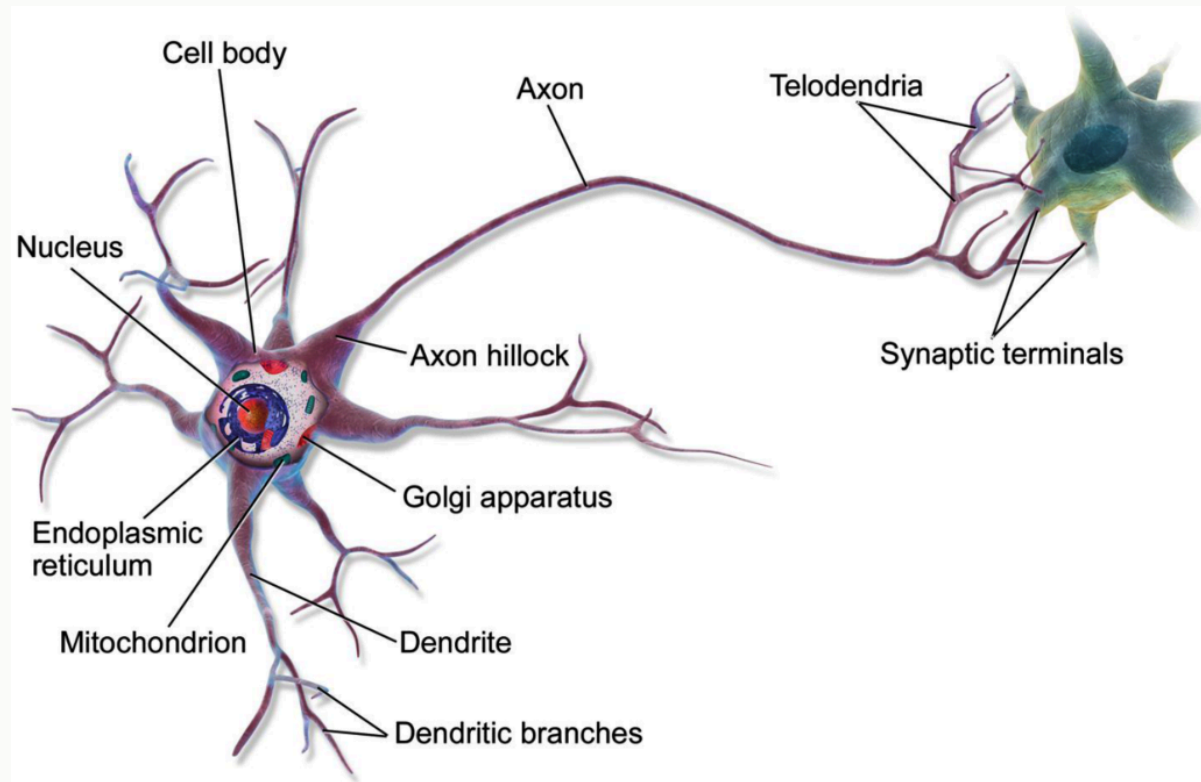


神經網路的發展與適用條件

- 1943 年的邏輯神經元、1957 年的 Perceptron，先建立簡單計算模型。
- 多層網路與反向傳播讓中間表示可以由資料學習。
- 大量資料、矩陣計算能力與訓練方法的改善，使更大模型變得可行。

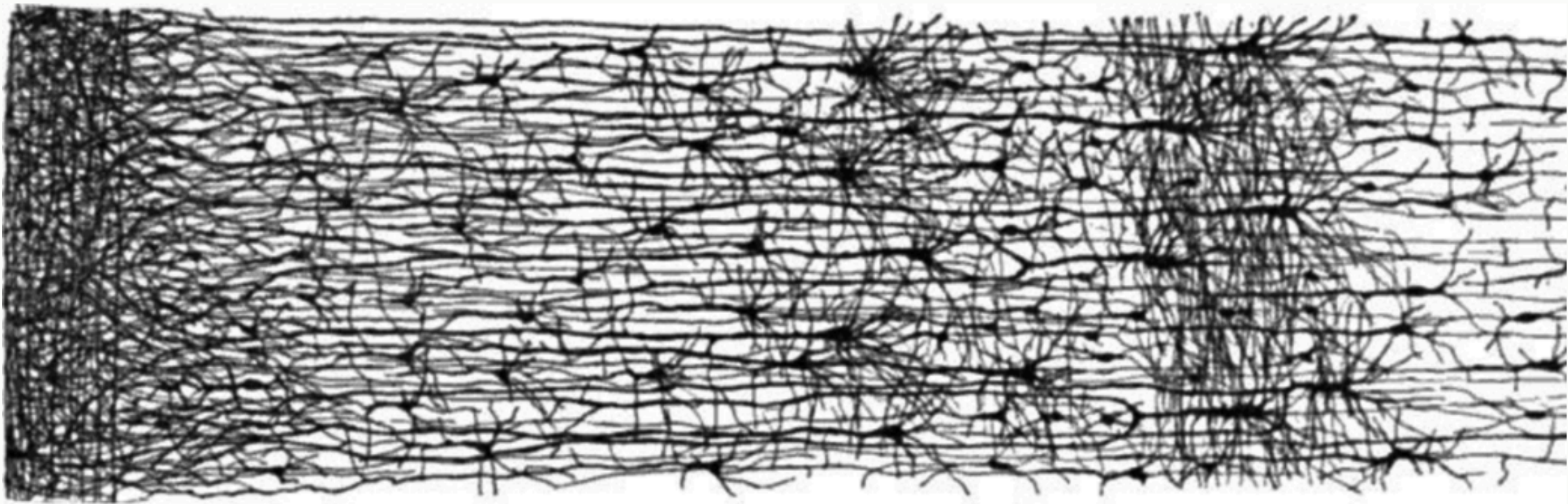
能表示複雜函數，不保證容易訓練，也不保證小型表格資料勝過線性模型或森林。後續再討論預訓練、遷移學習與 Transformer。

從生物神經元得到啟發



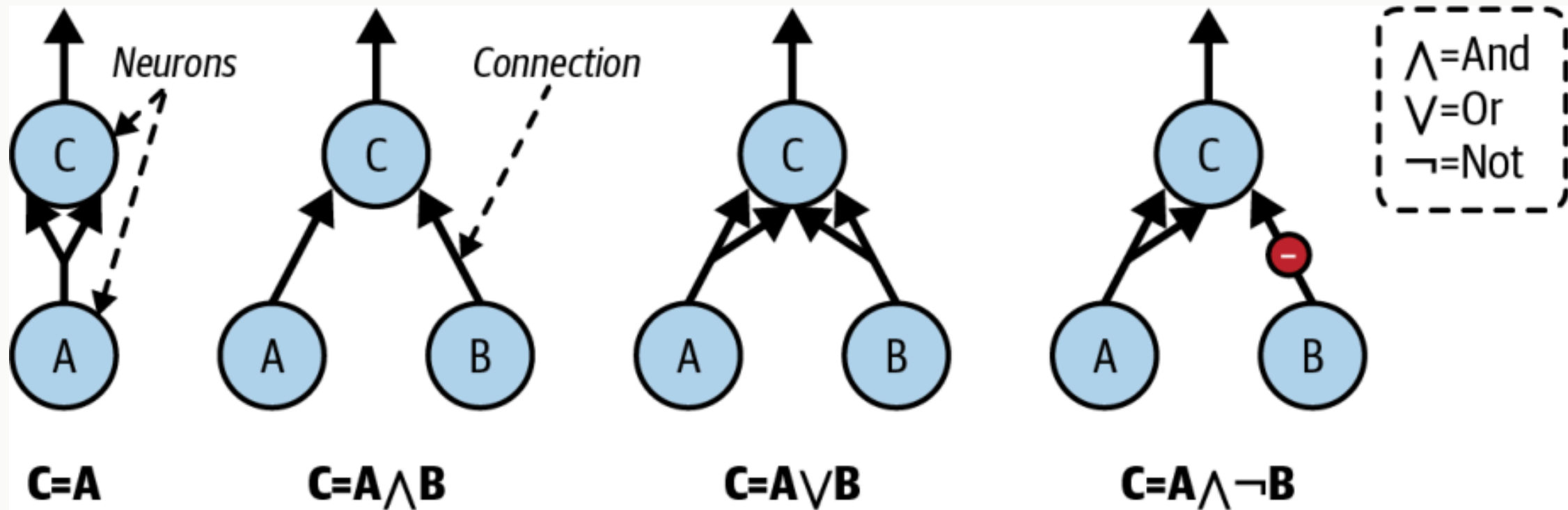
人工神經元是數學模型；樹突、細胞體與軸突提供概念類比，不等於完整腦科學模擬。

很多單元組合，形成更複雜的處理



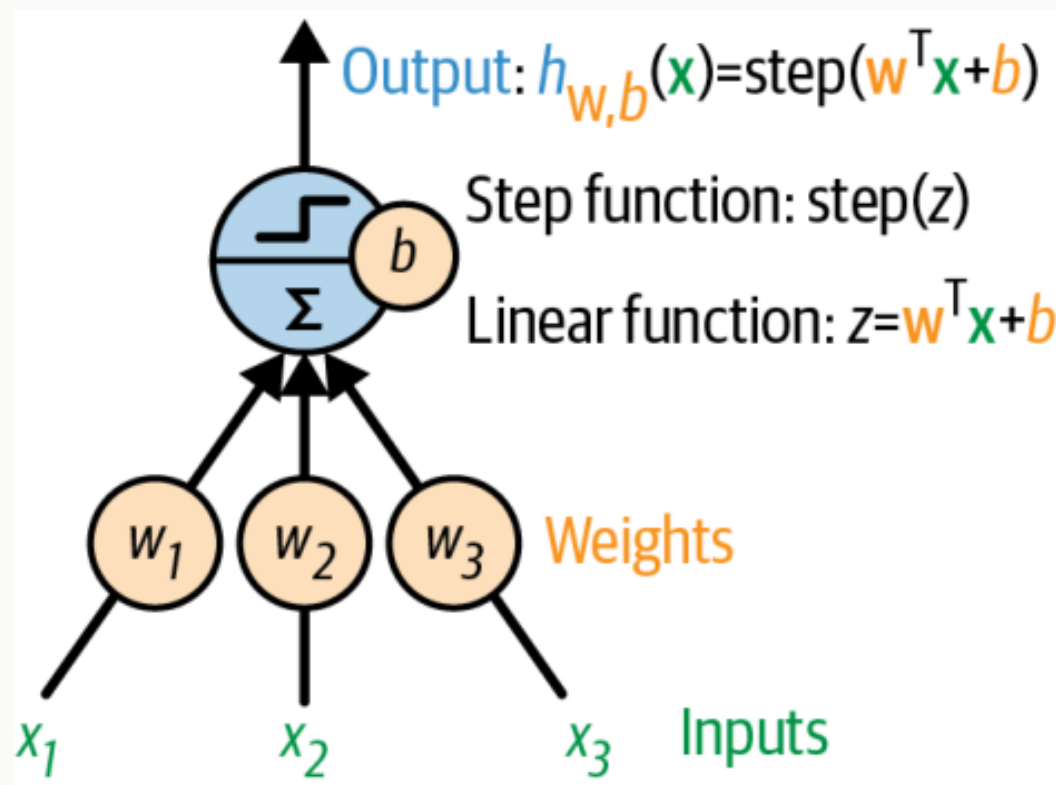
連接方式與權重決定訊息如何傳遞；學習是根據資料調整這些參數。

人工神經元可組合簡單邏輯



AND、OR、NOT 等邏輯可由不同連接與閾值表達；先觀察輸入如何決定輸出。

TLU：加權和，加偏置，再套階躍函數



輸入可以是實數；使用階躍時，輸出仍是離散的 0/1 或 $-1/+1$ 。

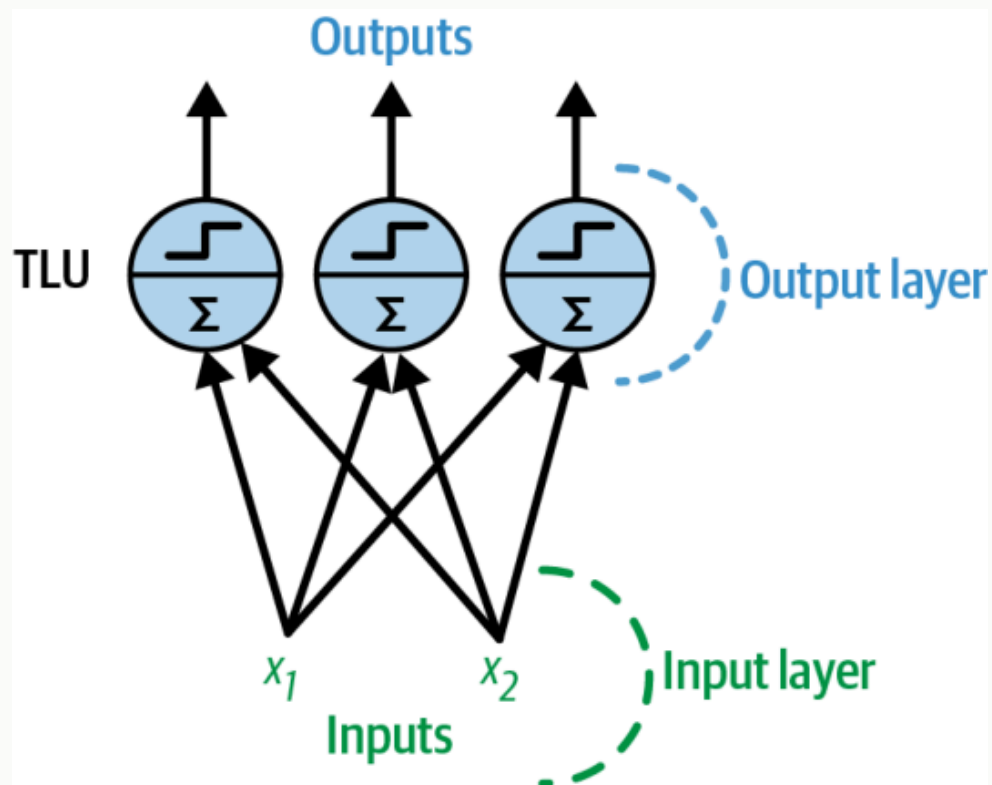
階躍函數與單層輸出

$$\text{step}(z) = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases}, \quad \text{sgn}(z) = \begin{cases} -1 & z < 0 \\ 0 & z = 0 \\ 1 & z > 0 \end{cases}$$

$$f(X) = \phi(XW + b) \circ$$

$X : m \times n$, $W : n \times q$, $b : q$; 偏置廣播到每列，輸出為 $m \times q$ 。

Perceptron 層：每個輸出都看所有輸入



全連接層有一組輸入到輸出的權重；偏置可視為固定輸入 1 的连接。

Perceptron 學習規則：答錯才修正

$$w_{i,j} \leftarrow w_{i,j} + \eta(y_j - \hat{y}_j)x_i$$

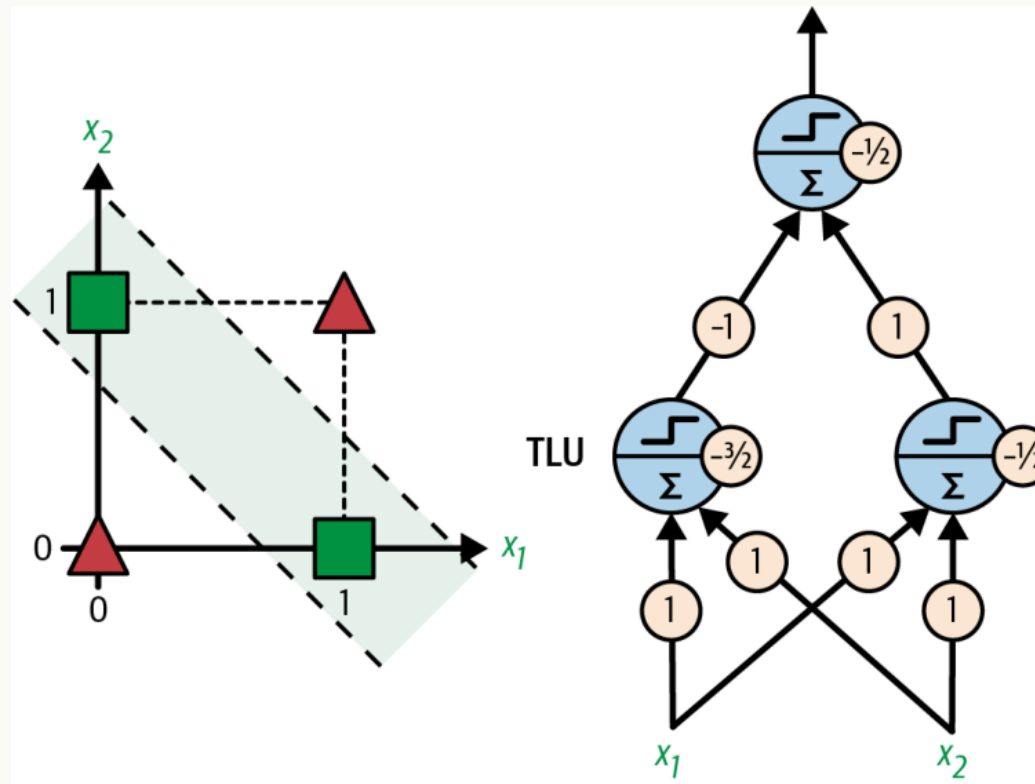
偏置同樣更新，等價令對應輸入為 1。線性可分資料可收斂到分隔面，但不保證最大間隔，也不直接輸出機率。

Perceptron 的學習限制

- 固定線性可分訓練資料時，經典更新可找到某個分隔面。
- 線性不可分時可能持續修正，實作需限制訓練輪數。
- 階躍函數幾乎處處導數為零，不能直接對它做一般梯度下降。

Perceptron 的錯分更新是一套學習規則；可微激活與損失，才讓後面的 MLP 能用反向傳播訓練。

XOR：一條線不夠，但組合多個單元可以



相同輸入位元輸出 0；不同輸出 1。隱藏層先建立中間邏輯特徵。

XOR 真值表：把每一列實際走過網路

A	B	A OR B	A AND B	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

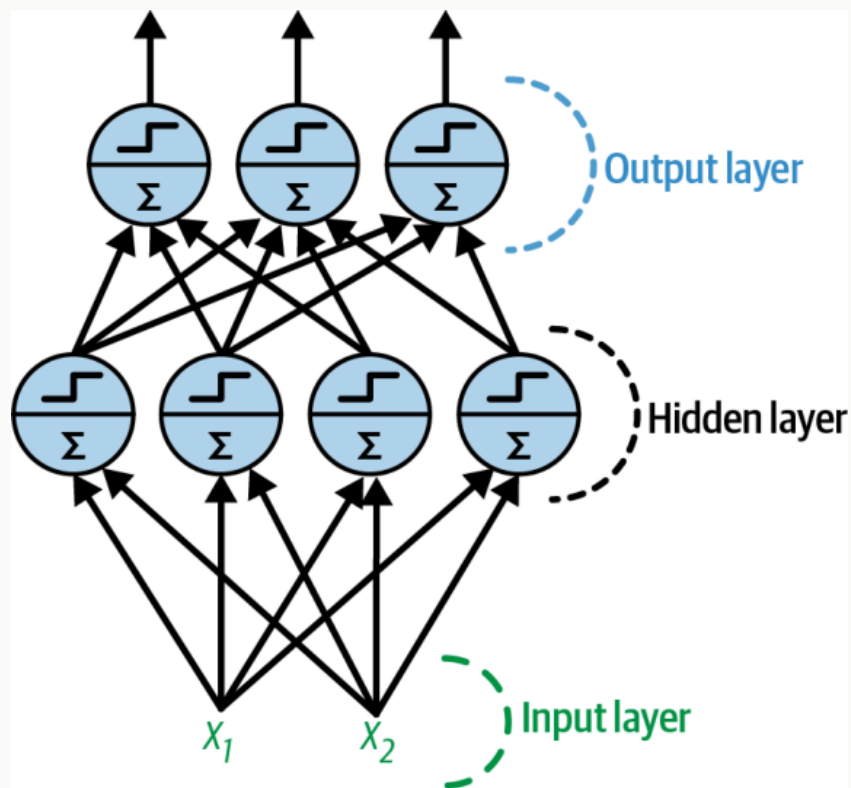
手算 XOR 網路的四組輸入

$$h_1 = \text{step}(A + B - 1.5), h_2 = \text{step}(A + B - .5)。$$

$$\hat{y} = \text{step}(-h_1 + h_2 - .5)。$$

代入四組輸入，確認輸出與真值表一致。再說明隱藏層增加了什麼能力。

MLP：輸入、隱藏層與輸出層



前饋網路的訊號向前流動；每層產生下一層的特徵表示。

MLP 形狀檢核：每層如何連接？

若輸入有 4 個特徵、隱藏層 3 個神經元、輸出層 2 個神經元：

- 第一層權重可寫成 $W_1 \in \mathbb{R}^{4 \times 3}$ ，偏置 $b_1 \in \mathbb{R}^3$ 。
- 第二層權重為 $W_2 \in \mathbb{R}^{3 \times 2}$ ，偏置 $b_2 \in \mathbb{R}^2$ 。
- 單筆輸入的路徑是 $x(1 \times 4) \rightarrow h(1 \times 3) \rightarrow y(1 \times 2)$ 。

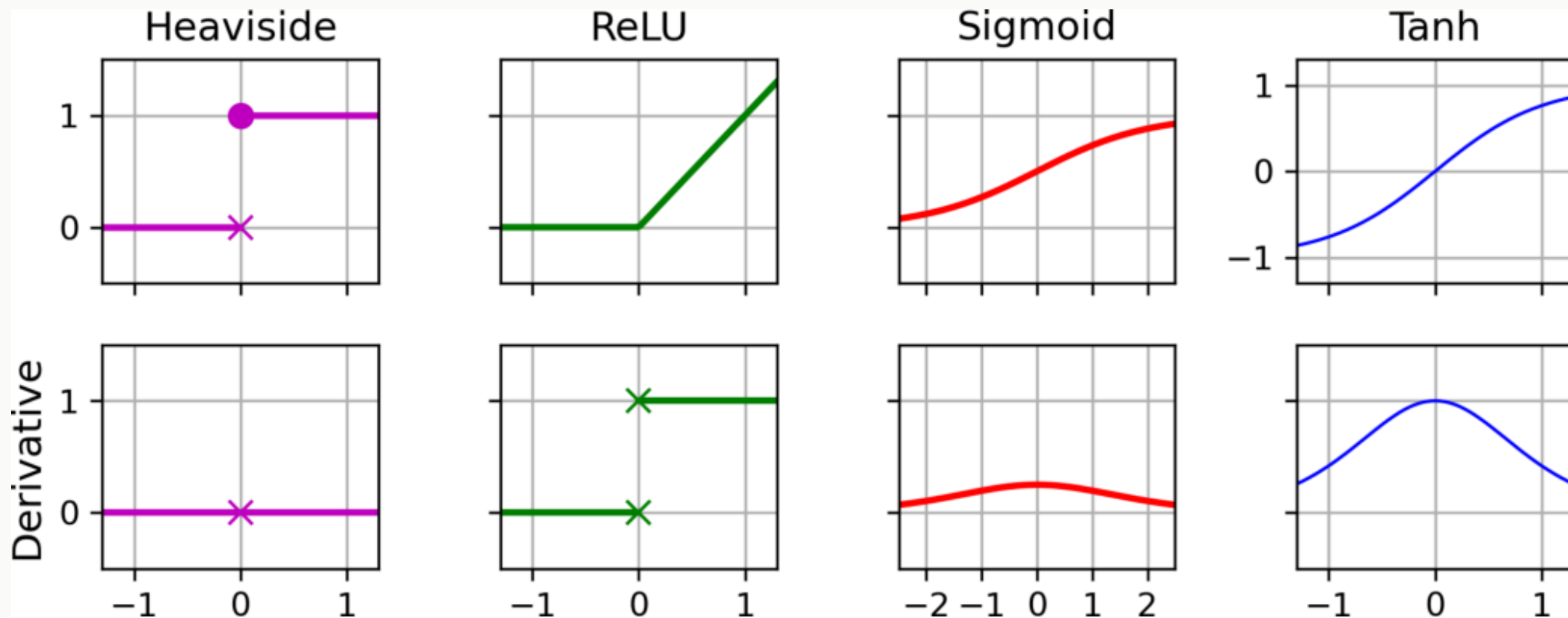
檢查形狀可及早找出矩陣相乘與廣播錯誤。

沒有非線性，堆再多層也會折成一層

$$f(g(x)) = (xW_1 + b_1)W_2 + b_2 = x(W_1W_2) + (b_1W_2 + b_2)$$

因此需要非線性激活函數，才能讓深度帶來新的表達能力。

Sigmoid、tanh、ReLU 與它們的導數



Sigmoid／tanh 的飽和區梯度很小；ReLU 的負半軸梯度為零。

激活函數：用公式連結曲線與梯度

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\tanh'(z) = 1 - \tanh^2(z), \quad \text{ReLU}(z) = \max(0, z)$$

ReLU 在 $z > 0$ 導數為 1， $z < 0$ 為 0；零點採實作約定。

選讀 | Leaky ReLU、ELU 與平滑激活

函數	定義	觀察重點
Leaky ReLU	$\max(\alpha z, z), 0 < \alpha < 1$	負半軸仍有小斜率
ELU	$z > 0$ 用 z ，否則 $\alpha(e^z - 1)$	負值區平滑且會飽和
Softplus	$\log(1 + e^z)$	平滑、嚴格正值的輸出
GELU	$z\Phi(z)$	用標準常態累積函數作平滑門控

函數形狀會影響梯度；是否改善訓練仍需比較驗證結果與成本。

選讀 | Swish 與 Mish 的門控形式

$$\text{Swish}(z) = z\sigma(\beta z), \quad \text{Mish}(z) = z \tanh(\text{softplus}(z))$$

$\beta = 1$ 的 Swish 也稱 SiLU。兩者都允許部分負值，並以平滑函數調整輸入幅度。

閱讀激活函數與導數圖，選 $z = -2, 0, 2$ 比較函數值及局部斜率；不要只因函數較新就判定更適合資料。

一次訓練步驟要分清楚四件事

1. 前向：計算各層輸出，保存中間值。
2. 損失：比較預測與真值。
3. 反向：用鏈式法則計算每個參數的梯度。
4. 更新：optimizer 根據梯度調整參數。

鏈式法則：沿路相乘，分支相加

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}$$

$z = wx + b$ 、 $a = \sigma(z)$ 。先計算靠近損失的梯度，再往前傳。

一個參數若影響多條路徑，將各路徑貢獻相加。

手算範例：一個 Sigmoid 神經元

$$x = 2, w = .5, b = 0, y = 1, L = \frac{1}{2}(a - y)^2 \circ$$

$$z = 1, \quad a = \sigma(1) \approx .731059, \quad L \approx .036165$$

$$\frac{\partial L}{\partial z} = (a - y)a(1 - a) \approx -.052877$$

接續手算：把梯度傳給權重與偏置

使用上一頁算得的 $\partial L / \partial z \approx -0.052877$ ，並令 $x = 2$ 。

1. 求 $\partial L / \partial w$ 、 $\partial L / \partial b$ 。
2. 用學習率 0.1 更新 w, b 。
3. 重算預測，判斷損失是否下降。

多層網路的形狀與反向遞迴

$$Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)}, \quad A^{(l)} = \phi_l(Z^{(l)})$$

$$\delta^{(l)} = (\delta^{(l+1)}W^{(l+1)T}) \odot \phi_l'(Z^{(l)})$$

$$\nabla_{W^{(l)}} L = A^{(l-1)T} \delta^{(l)}$$

本頁 δ 定義為平均損失對 Z 的梯度，已包含批次平均因子。

梯度消失與梯度爆炸

多層鏈式法則會連乘權重與激活導數。Sigmoid 的導數最多 $1/4$ ；若只看十層激活部分：

$$(1/4)^{10} \approx 9.54 \times 10^{-7}$$

這只示範縮小效應，完整梯度還包含權重與分支加總。過大的乘積也可能使梯度爆炸。

檢查各層梯度與損失，再調整激活、初始化與學習率；ReLU 並不保證消除所有梯度問題。

初始化：打破隱藏神經元的對稱

同層神經元若具有完全一樣的權重與偏置，收到相同梯度後仍會一樣。

隨機初始化權重讓它們學不同表示；偏置常可設為 0。

多層小導數相乘會讓梯度消失；初始化尺度、激活與網路深度需一起考慮。

為何隱藏神經元不能全部同值初始化？

若同層神經元的權重與偏置完全相同，前向輸出相同，反向收到的梯度也相同；更新後仍維持對稱。

這些神經元就難以學到不同特徵。通常隨機初始化權重以打破對稱，偏置則可初始化為零；還要選擇與激活函數相配的尺度。

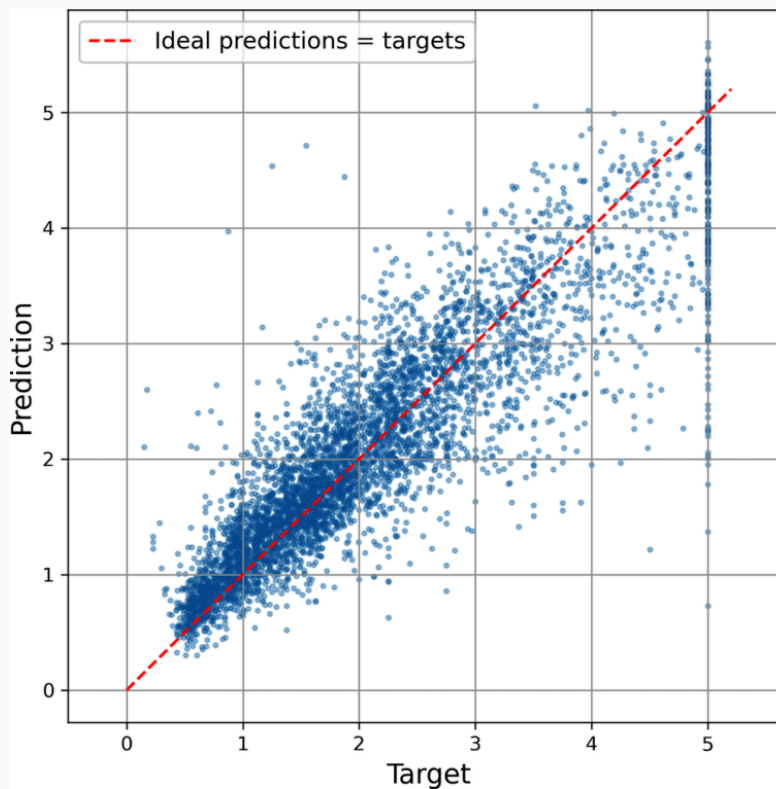
表 9-1：迴歸 MLP 的典型架構 (1/2)

項目	教材典型值
隱藏層數	依問題，常見 1–5
每隱藏層神經元	依問題，常見 10–100
輸出神經元	每個目標維度 1 個
隱藏層激活	ReLU

表 9-1：輸出範圍與損失 (2/2)

需求	輸出激活／損失
任意實數	線性輸出
非負／正值	ReLU／softplus
有界範圍	Sigmoid／tanh，配合目標縮放
迴歸損失	MSE；有離群值可考慮 Huber

MLP 迴歸：預測與真值散點



越接近對角線越好；還要看高目標值、殘差與單位。此例 RMSE 約 0.53；目標單位為十萬美元，約相當於 5.3 萬美元。

用 scikit-learn 建立小型迴歸 MLP

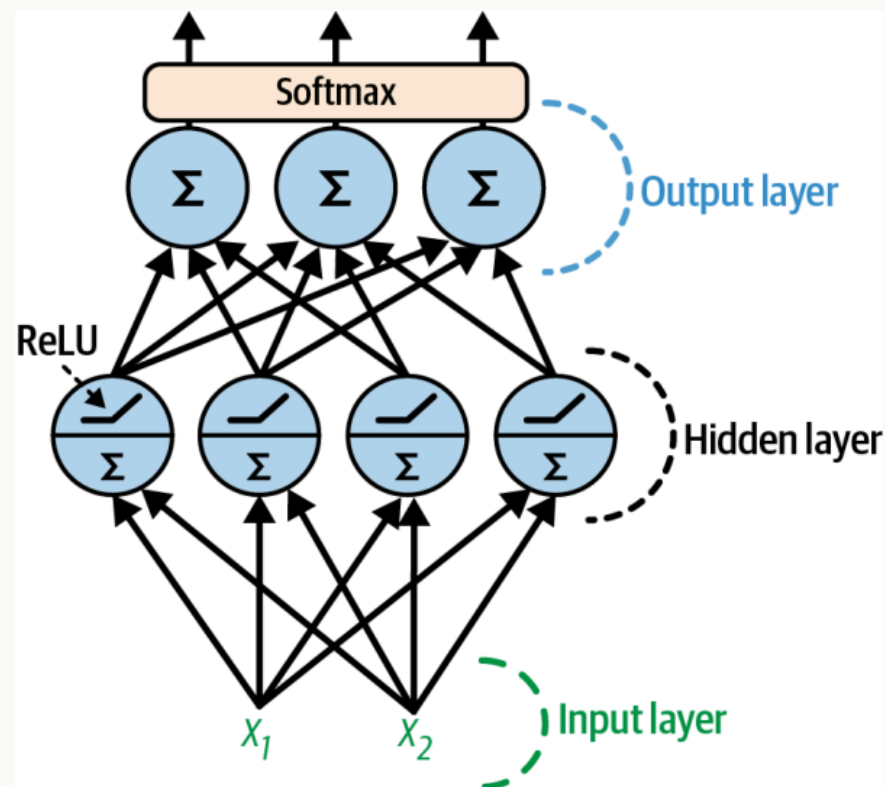
```
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
reg = make_pipeline(StandardScaler(), MLPRegressor(
    hidden_layer_sizes=(50, 50), early_stopping=True,
    max_iter=300, random_state=42))
reg.fit(X_train, y_train)
```

表 9-2：分類的輸出層與損失

共通設定：隱藏層通常 1-5 層，依任務調整。

任務	輸出個數	激活	損失
二元	1	Sigmoid	二元交叉熵
多標籤	每標籤 1	各自 Sigmoid	各標籤二元交叉熵
互斥多類別	每類 1	Softmax	多類別交叉熵

分類 MLP：隱藏層 ReLU，輸出層 Softmax



對互斥多類別產生總和為 1 的機率；這些機率仍可能過度自信。

Softmax 與交叉熵的輸出梯度

$$p_k = \frac{e^{z_k - \max_j z_j}}{\sum_j e^{z_j - \max_l z_l}}, \quad L = - \sum_k y_k \log p_k$$

當 $\sum_k y_k = 1$ ，對單筆資料 $\partial L / \partial z_k = p_k - y_k$ 。

批次平均損失的梯度再除以批次樣本數。這是特定激活與損失配對的結果，不能套用到所有輸出層。

選讀 | 原 MNIST 手刻網路的損失配對

NeuralNetMLP：784 輸入、50 個 Sigmoid 隱藏單元、10 個各自 Sigmoid 輸出。

若 $L = m^{-1} \sum_{i,k} (a_{ik} - y_{ik})^2$ ，則

$$\delta_{ik} = \frac{2}{m} (a_{ik} - y_{ik}) a_{ik} (1 - a_{ik})$$

這組輸出不保證總和為 1。改成 Softmax+交叉熵時，前向、損失與反向梯度要一起改，不能只換輸出激活。

Fashion MNIST：由像素到服飾類別



每張 28×28 灰階圖可攤成 784 個特徵；10 種類別比單純數字更有外觀變異。

分類 MLP 的訓練與預測流程

```
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import MinMaxScaler
clf = make_pipeline(MinMaxScaler(), MLPClassifier(
    hidden_layer_sizes=(100, 50), early_stopping=True,
    max_iter=200, random_state=42))
clf.fit(X_train, y_train)
pred, proba = clf.predict(X_test), clf.predict_proba(X_test)
```

影像先攤平；類別標籤用一維整數。預測仍經過同一 Pipeline，避免漏掉縮放。

訓練損失、驗證分數與測試指標不能混用

- MLPRegressor 的 `score` 是 R^2 ; RMSE 需另算。
- MLPClassifier 的 `score` 是 accuracy ; log loss 另算。
- `loss_curve_` 記錄最佳化損失，可能包含正則化項。
- 早停監控的是內部驗證分數，不是最終測試結果。

Epoch、批次與早停的資料角色

一個 epoch 看完訓練資料一次。1,000 筆資料、batch size=128，若保留尾批，每個 epoch 有 8 次更新。

`max_iter` 對 SGD／Adam 訓練表示最大 epoch 數；batch size 影響每次更新的樣本數。

`early_stopping=True` 在傳入的訓練資料內留出驗證部分，依分數改善與耐心值停止。最終測試集只在選模完成後使用。

教材案例提醒：高信心可能仍然答錯

教材報告測試 accuracy 約 87.1%，並展示高信心錯誤；但示例預測略過縮放 Pipeline，不能直接歸因於模型的校準問題。

先確認訓練與預測使用相同前處理，再檢查錯誤案例、log loss 與校準；一般框架可考慮 label smoothing。

scikit-learn 的 MLP 不直接提供完整的 label smoothing、dropout 或 TensorBoard 訓練介面。

調參次序：先讓模型學得動，再增加容量

檢查	可調項目	看什麼證據
資料尺度與切分	StandardScaler、資料角色	是否洩漏、分布是否不同
最佳化	學習率、batch size、optimizer	收斂曲線、警告、時間
容量	隱藏層數、每層神經元	訓練與驗證差距
正則化	alpha、早停	是否減少驗證誤差

深度、寬度與遷移學習

較深網路可重複組合中間特徵，某些函數可比淺層表示更有效率。

過窄的中間層可能形成資訊瓶頸；過多參數則增加訓練與資料需求。

遷移學習重用已有表示；本週先完成小型 MLP，後續再進入 PyTorch 與 GPU。

學習率、批次與參數量的調整

- 每層參數數量為 $(n_{\text{in}} + 1)n_{\text{out}}$ 。784→300→100→10 共 **266,610** 個參數。
- 學習率太小進展慢，太大可能震盪；先以對數尺度比較候選值。
- 大批次可減少每個 epoch 的更新次數，但吞吐量與泛化未必一起改善。
- 改變 batch size 或 optimizer 後，要重新檢查學習率與驗證曲線。

同寬隱藏層是合理起點，不必每層都縮成金字塔；瓶頸也可能刻意用來學壓縮表示。

選讀 | 學習率範圍測試

短暫試跑時，從小學習率開始，以固定倍率增加，記錄「學習率對損失」：

$$\eta_t = \eta_0 q^t, \quad q = (\eta_{\text{end}} / \eta_0)^{1/T}$$

觀察快速下降與開始發散的區域，選較保守的候選值，再以驗證集確認。正式訓練須重設模型與最佳化器狀態。

試跑是選參數的工具，不能把「發散點的一半」當成所有模型通用的最佳值。

選讀 | Dropout 的訓練與推論

訓練時以機率 p 暫時遮掉部分隱藏激活，讓模型減少依賴固定幾個單元。常見 inverted dropout：

$$\tilde{a} = \frac{r \odot a}{1 - p}, \quad r_j \sim \text{Bernoulli}(1 - p)$$

所以 $E[\tilde{a}_j] = a_j$ ；一般推論關閉隨機遮罩，直接使用完整激活。

Dropout 不是永久刪除神經元；比例需驗證，也不保證提升固定百分比。sklearn MLP 沒有 Dropout 層設定。

選讀 | 訓練紀錄、早停與模型回復

工具／步驟	保存或觀察的內容
訓練紀錄	每輪 train loss、validation loss／指標與學習率
TensorBoard	讀取各次實驗的 event files，比較曲線與模型圖
Checkpoint	保存驗證最佳的參數，供停止後回復
重載檢查	同時保存前處理、類別順序與模型設定，核對預測

早停控制何時結束，Checkpoint 決定保留哪一版模型。測試指標不作反覆選版本的依據。

選讀 | Keras 與 PyTorch 的概念銜接

Keras 案例	核心概念	PyTorch 對應
Sequential、回歸／分類	逐層前向、損失、更新、評估	Ch.10 基本模型與訓練迴圈
Functional、Subclassing	自訂前向圖，多輸入／多輸出	Ch.10 自訂 Module
Wide & Deep	原始特徵支路與深層特徵串接	Ch.10 非序列模型
搜尋、保存與 callbacks	驗證選參數、保存最佳狀態	Ch.10 調參及模型保存

本週不要求安裝 Keras。先完成 sklearn MLP；框架實作與深層訓練接在 Ch.10、11。

綜合實作：小型 MLP 與簡單模型比較

用已備妥的 digits，小型 MLP (32,) 與 (32,16) 各跑一次；與標準化 Logistic 比較。

提交：資料切分、accuracy、log loss、訓練時間、兩個錯誤案例。

離線組：10 輸入→50 隱藏→3 輸出的網路，列出各矩陣形狀並計算參數數目。

離堂檢核：把八週串回同一個學習流程

1. Backprop 計算什麼？Optimizer 改變什麼？
2. 二元、多類別、多標籤的輸出為何不同？
3. 為何深度模型仍需與簡單基準比較？

課後：Ch.9 練習 2-9；練習 1 的 playground 與練習 10 大型資料作延伸。

Notebook 導讀：架構、梯度與成效分開檢查

對照內容	Notebook 的可觀察證據
TLU、XOR、MLP 架構	真值表、決策區域、各層矩陣形狀
反向傳播與輸出損失	單神經元手算、有限差分、 $p - y$
迴歸與分類	真值／預測散點、digits 高信心錯誤
訓練控制	學習率尺度、Dropout 期望、獨立最佳快照

手刻 MLP 的損失是每筆平方誤差「加總輸出、平均樣本」。
生物圖僅作背景；Notebook 使用 digits 示範影像分類流程。